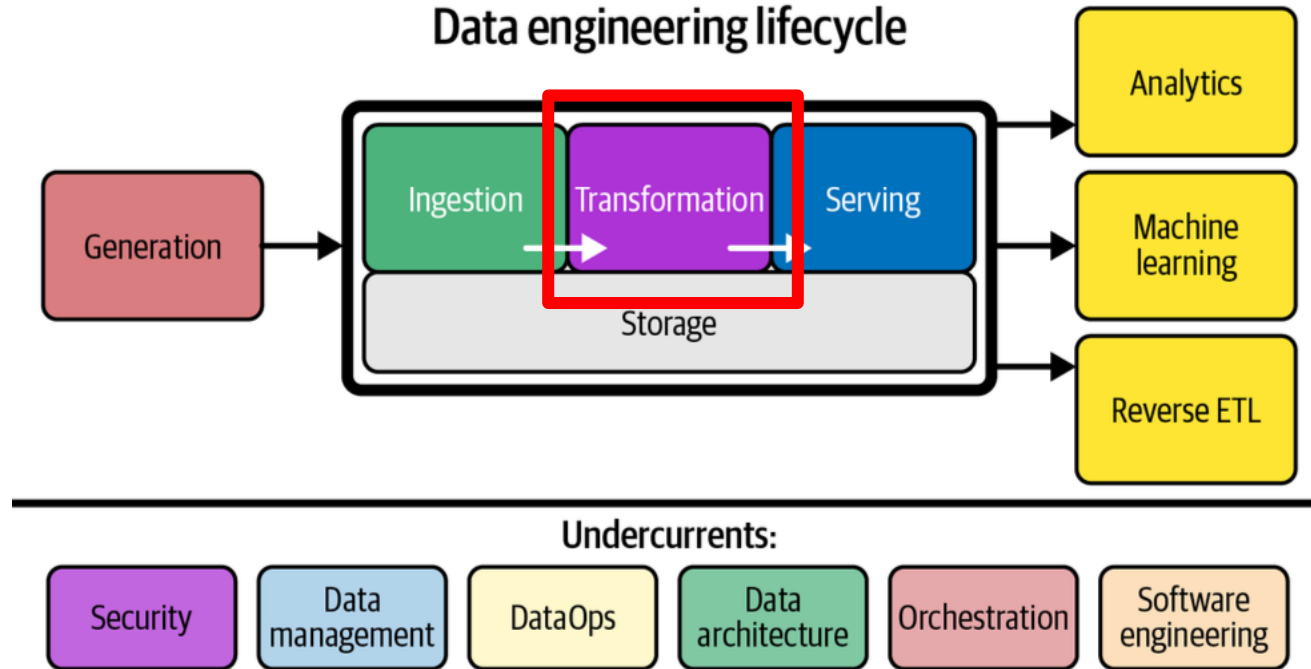


Data Engineering Transformation

UA.DETI.ED - 2025/26

José Luís Oliveira

The Data Engineering Lifecycle



Queries, Modelling, and Transformation

❖ Creating Value from Raw Data

- This stage of the lifecycle is where data is made useful for downstream stakeholders by turning raw ingredients into consumable information.

❖ Transformation

- Transformations sit centrally between the ingestion and serving stages within the data engineering lifecycle.

❖ Languages

- While many languages exist, SQL remains the most popular and universal query language for these tasks in both OLAP and NoSQL environments.

Defining the Query

❖ The Foundation of Data Retrieval

- A query allows you to retrieve and act on data, representing the "Read" in the CRUD (Create, Read, Update, Delete) paradigm.
- Queries utilize predicates or logical conditions to filter specific records from a dataset.

❖ Essential SQL Sub-Languages

- **Data Definition Language** (DDL) defines database objects like schemas and tables using commands such as CREATE, DROP, and ALTER.
- **Data Manipulation Language** (DML) is used to add or change data within objects through commands like SELECT, INSERT, UPDATE, DELETE, and MERGE.

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY,  
    FirstName VARCHAR(50),  
    LastName VARCHAR(50),  
    Department VARCHAR(50)  
);
```

```
SELECT FirstName, LastName  
FROM Employees  
WHERE Department = 'Sales';
```

Query Control and Transactions

❖ Data Control Language (DCL)

- DCL allows engineers to control access to database objects through commands like GRANT, DENY, and REVOKE.
 - GRANT SELECT ON data_science_db To user user_name Maria;
- This is essential for security-minded engineers to ensure users have only the necessary access to specific databases or tables.

❖ Transaction Control Language (TCL)

- TCL manages the details of transactions, defining commit checkpoints and conditions for rolling back changes.
- Common commands include COMMIT and ROLLBACK, which ensure the database maintains a consistent state during complex operations.

The Life of a Query

❖ From Code to Results

- The database engine first parses the SQL to check for proper semantics and ensure the user has appropriate access rights.
- The SQL code is converted into bytecode, an efficient machine-readable format for the database engine.

❖ Optimization and Execution

- The query optimizer analyzes the bytecode to reorder and refactor steps for maximum efficiency.
- Finally, the query is executed against the storage engine, and results are produced for the user.

Improving Query Performance

❖ Optimizing Join Strategies

- Joins are critical but compute-intensive; performance can be improved by prejoining frequently used datasets into **wide tables** or **materialized views**.
- Engineers must be cautious of "row explosion," which occurs during many-to-many matches and can consume massive system resources.

❖ Advanced SQL Techniques

- Using **Common Table Expressions** (CTEs) instead of nested subqueries improves code readability and often delivers better performance.
- Explain plans (**EXPLAIN** command) allow engineers to see exactly how the optimizer will execute a query and identify bottlenecks.

Nested Subquery vs CTE

```
SELECT r.RepName, TotalSales.RepTotal
FROM Reps r
JOIN (
    -- First subquery calculates total sales per rep
    SELECT RepID, SUM(Amount) as RepTotal
    FROM Sales
    GROUP BY RepID
) AS TotalSales ON r.RepID = TotalSales.RepID
WHERE TotalSales.RepTotal > (
    -- Second subquery calculates the overall average
    SELECT AVG(RepTotal)
    FROM (
        SELECT SUM(Amount) as RepTotal
        FROM Sales
        GROUP BY RepID
    ) AS AvgSales
);
```

```
-- 1. Define total sales per rep once
WITH RepTotalSales AS (
    SELECT RepID, SUM(Amount) AS RepTotal
    FROM Sales
    GROUP BY RepID
)

-- 2. Use the CTE in the main query
SELECT r.RepName, rts.RepTotal
FROM Reps r
JOIN RepTotalSales rts ON r.RepID = rts.RepID
WHERE rts.RepTotal > (
    -- 3. Reuse the CTE to find the average
    SELECT AVG(RepTotal) FROM RepTotalSales
);
```

Storage-Level Query Tuning

❖ Avoiding Full Table Scans

- `SELECT *` without predicates is highly inefficient and expensive in pay-as-you-go cloud environments.
- Pruning reduces scanned data by selecting only required columns in columnar databases or using partitions and cluster keys for large datasets.

❖ Database Maintenance

- Vacuuming dead records is necessary to remove outdated pointers created during updates and deletes.
 - **Vacuuming** is the process of permanently removing dead records.
- Regular vacuuming prevents table bloat, keeps query plans accurate, and improves index performance.
 - `VACUUM`;
 - `VACUUM Employees`;

Consistency and Caching

❖ Understanding Commits

- Commits ensure isolation and a consistent state during concurrent reads and writes.
- Point-in-time snapshot models allow long-running queries to see a consistent picture without locking tables.

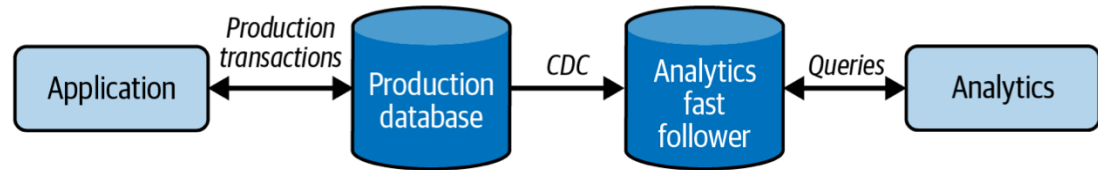
❖ Leveraging Result Caching

- Cold queries run against storage can be slow and expensive; many OLAP databases cache results for instant retrieval on identical subsequent requests.
- Caching reduces the computational burden on the database and provides a better experience for end users.

Queries on Streaming Data

❖ Real-Time Paradigms

- The "fast-follower" approach uses CDC to replicate a production database to an analytics system, allowing real-time queries without impacting application performance.
- Some systems use a streaming buffer combined with columnar storage to support interactive analytics on current data.



❖ Kappa Architecture

- This architecture treats all data as events and stores them in a streaming system for long-term retention and historical querying.
- Tools like Kafka KSQL allow for direct aggregation and statistical calculations on these persisted event streams.

Windowing in Streams

❖ Dynamic Time-Based Processing

- Windows are small batches processed based on triggers, such as session, fixed-time (tumbling), and sliding windows.
- Session windows group events by periods of activity per key (e.g., user ID), closing after a specified inactivity timeout.

❖ Fixed and Sliding Windows

- Fixed-time (tumbling) windows process data on a rigid schedule, such as every 20 seconds, to emit roll-up statistics.
- Sliding windows can overlap, such as generating a new 60-second window every 30 seconds for continuous metric updates.

Stream Joins and Watermarks

❖ Handling Temporal Complexity

- Watermarks serve as thresholds to determine if incoming data is within an acceptable window or is considered "late-arriving".
- Enrichment involves joining an incoming stream to static or semi-static data (like a cache or lookup table) to provide context.

❖ Stream-to-Stream Joins

- These joins are complex due to varying latencies between streams; they typically require a buffer to hold events until matching data arrives.
- Events are joined within these buffers and eventually evicted once the retention interval has passed.

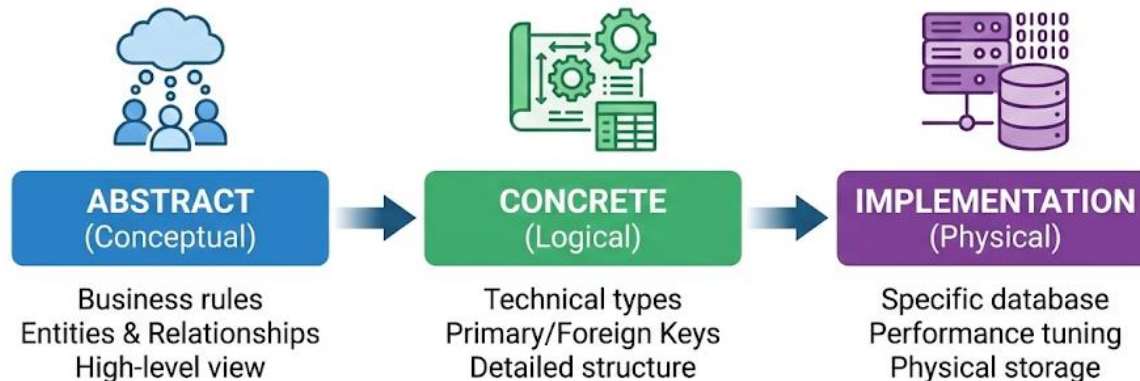
Foundations of Data Modeling

❖ Data modeling

- The process of choosing a structure that reflects an organization's processes and business logic.
- A good model provides consistent definitions across the company, ensuring that terms like "customer" mean the same thing to everyone.

❖ The Modeling Continuum

- Modeling moves from abstract to concrete



Modeling Batch Analytical Data

Inmon: The Top-Down Approach

❖ The Corporate Information Factory

- Bill Inmon's approach focuses on a subject-oriented, integrated, and nonvolatile collection of granular corporate data.
- Data is ETLed from source systems into a highly normalized (3NF) central data warehouse to ensure a "single physical corporate image".

❖ Data Marts

- The centralized warehouse serves as the single source of truth, from which department-specific data marts are built for reporting.
- This structure provides strong integration but can be more complex to implement and maintain.

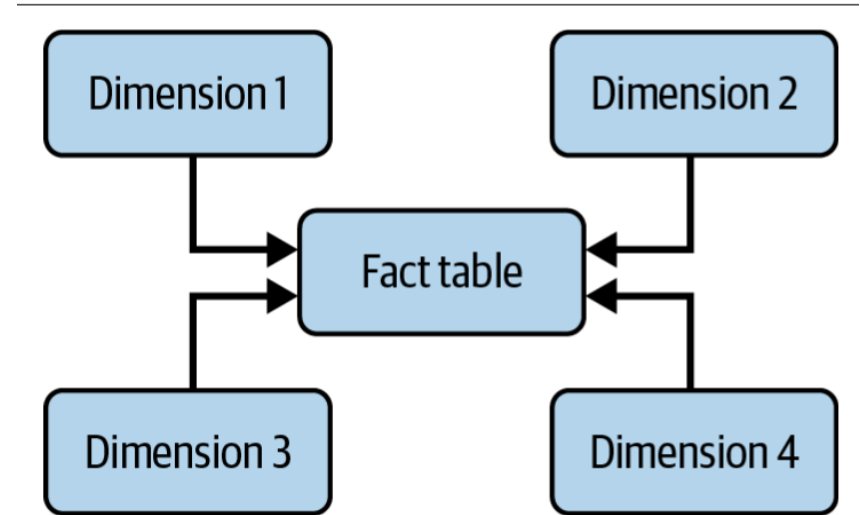
Kimball: The Bottom-Up Approach

❖ Dimensional Modeling

- Ralph Kimball's approach focuses on ease of use and query performance by using facts and dimensions in a star schema.
- It effectively makes the data mart the data warehouse itself, allowing for faster iteration than the Inmon approach.

❖ The Star Schema

- A central Fact table (quantitative events) is surrounded by Dimension tables (descriptive attributes).
- Dimensions are often denormalized, and data redundancy is accepted to simplify joins and improve query speed.



Facts and Dimensions

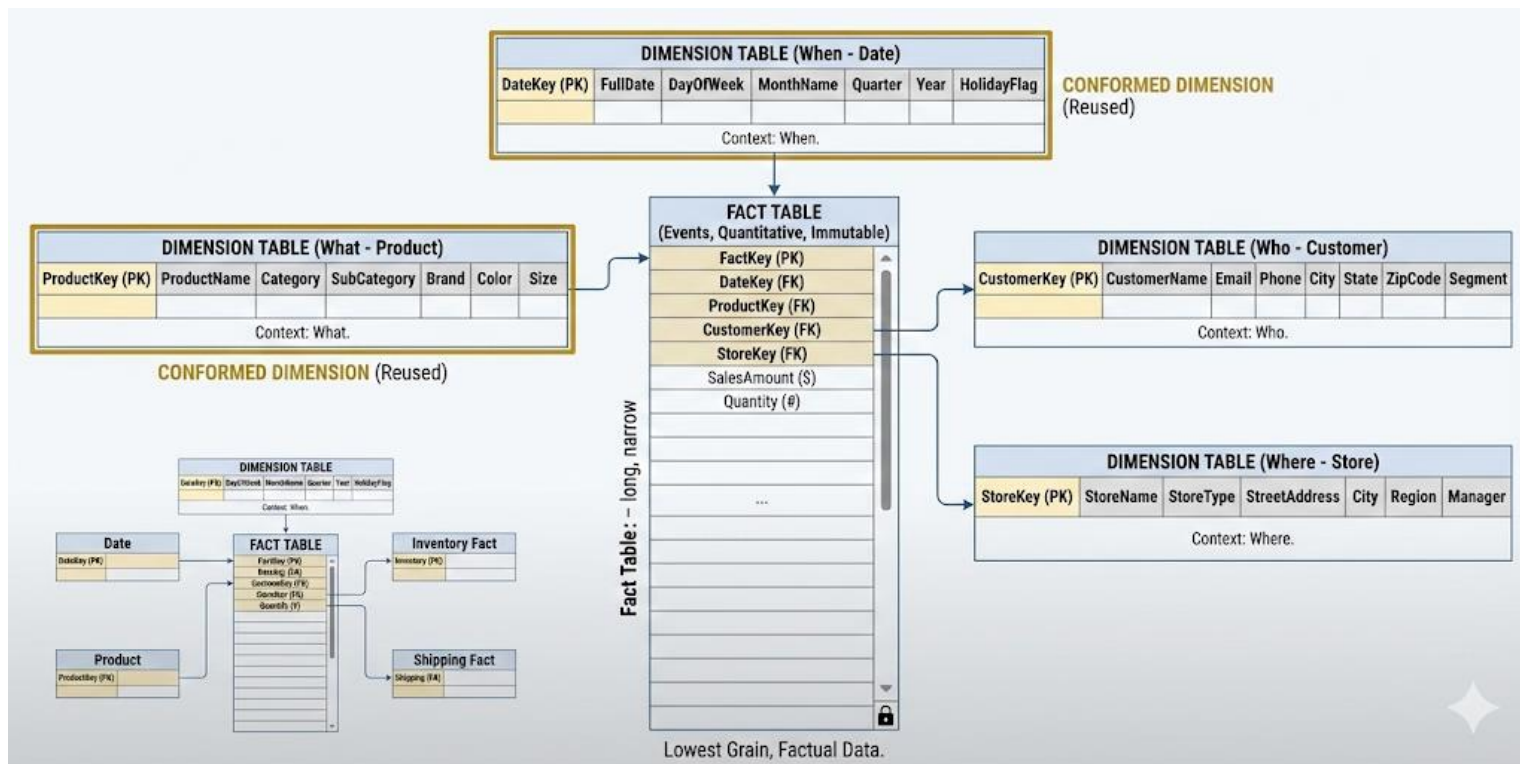
❖ Fact Tables

- Fact tables contain factual, quantitative data about events; they are typically long, narrow, and immutable.
- They should be kept at the lowest possible grain and contain keys to reference surrounding dimensions.

❖ Dimension Tables

- Dimensions provide the relational context (who, what, where, when) for events and are typically short and wide.
- Conformed dimensions are those reused across multiple star schemas to ensure consistency across the business.

Facts and Dimensions: Example



Slowly Changing Dimensions (SCD)

❖ Tracking Historical Changes

- SCDs are necessary to manage changes in dimension attributes over time while maintaining historical accuracy.

❖ Type 1

- Simply overwrite old data, meaning historical context is lost.
 - If John Smith changes his name to John Doe, the database record is updated from "Smith" to "Doe."

❖ Type 2 (most common)

- Insert a new row for every change with effective start and end dates to maintain full history.

❖ Type 3

- Adds a new column to store the "previous" value, allowing for a limited view of history.
 - Instead of adding new rows (Type 2), we alter the table structure by adding columns. A typical customer table might have columns for *Current_Region* and *Previous_Region*.

Data Vault Modeling

❖ Agile and Flexible Architecture

- The Data Vault separates structural aspects of data from its descriptive attributes.
- It is an insert-only methodology designed to be highly scalable and adaptable as business data evolves.

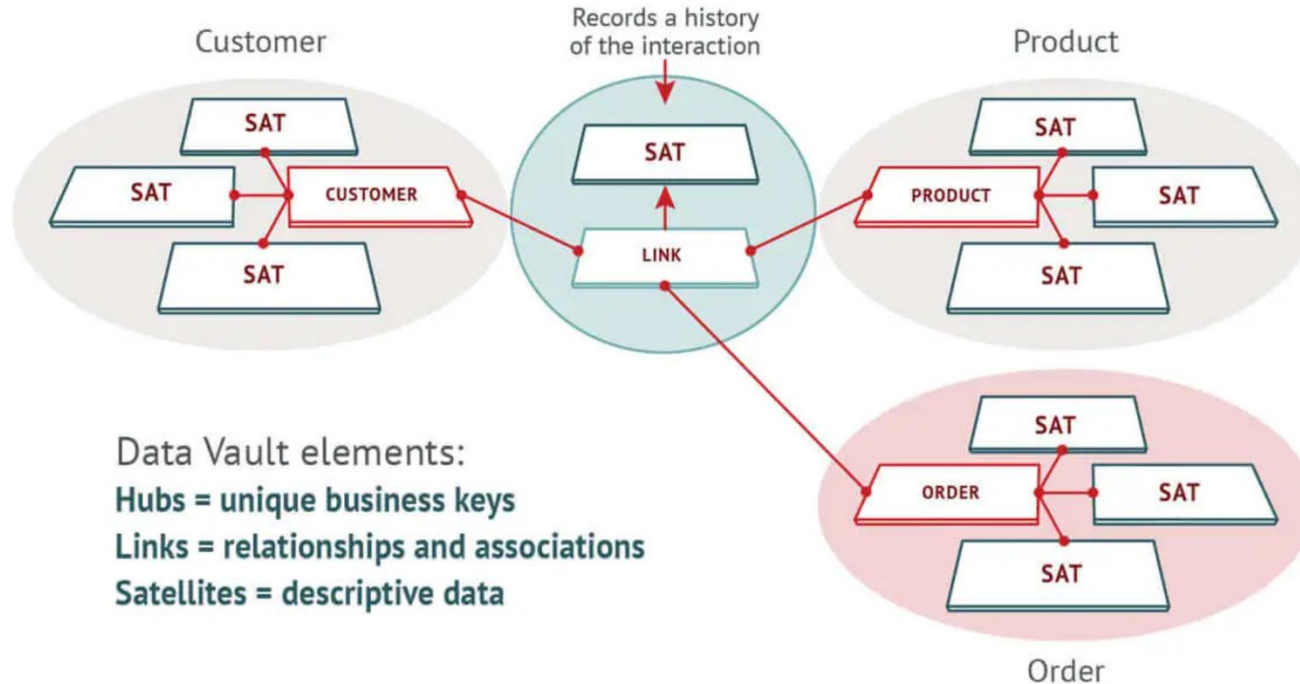
❖ Three Core Components

- **Hubs** store unique business keys;
- **Links** track relationships between these keys;
- **Satellites** store descriptive attributes and context.



- ❖ Unlike Kimball or Inmon, business logic is applied when the data is queried rather than during the loading process.

Data Vault Modeling



Source: wherescape.com

Wide Denormalized Tables

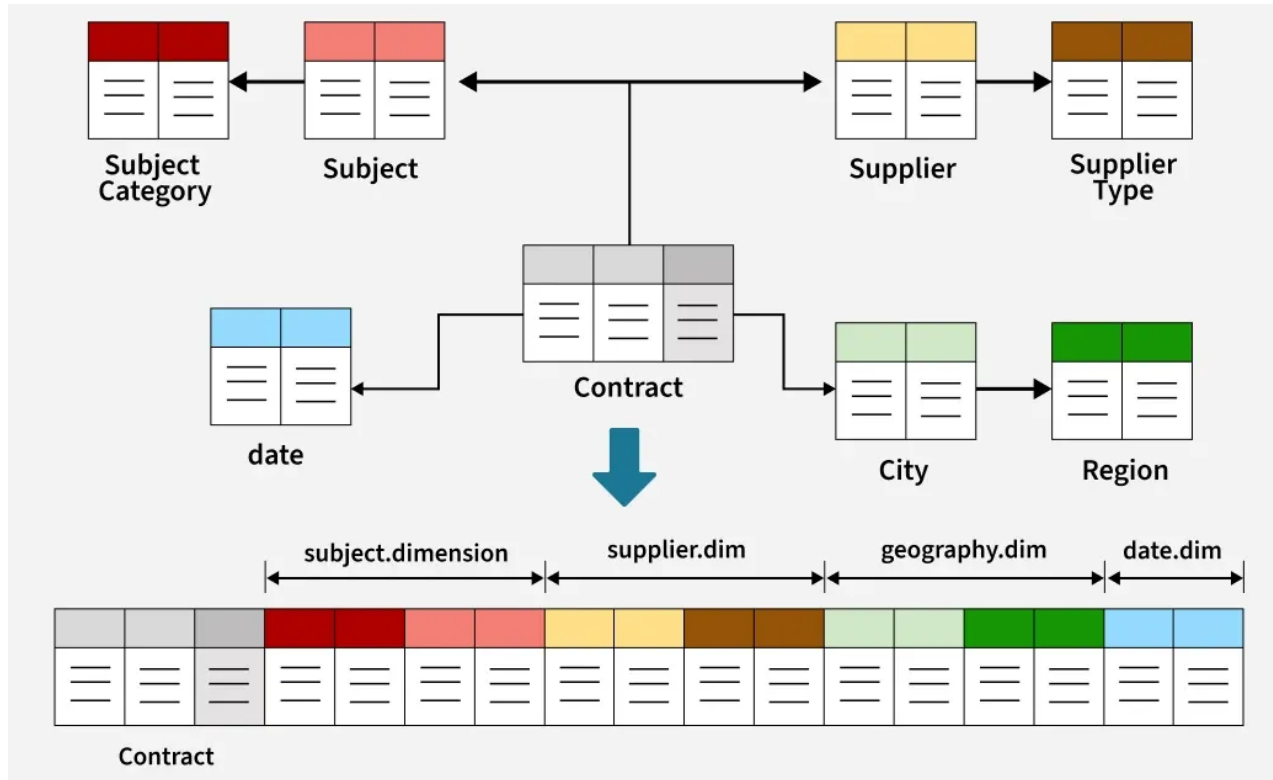
❖ Modeling for the Cloud Era

- Wide tables combine hundreds or thousands of columns into a single, highly denormalized table.
- This approach is made possible by cheap cloud storage and the efficiency of columnar databases at handling sparse data and nulls.

❖ Performance and Trade-offs

- Queries often run faster because joins are eliminated, but the lack of rigor can lead to lost business logic and difficult updates.
- This is an ideal pattern for fast-moving streaming data and flexible schemas where traditional modeling might be too slow.

Wide Denormalized Tables



<https://www.geeksforgeeks.org/dbms/denormalization-in-databases/>

Transformations

Transformations Defined

❖ Unifying and Integrating Data

- Transformations manipulate and enhance data to increase its value for downstream users.
- They differ from queries in that they persist results (temporarily or permanently) and handle much higher levels of complexity.

❖ The Role of Orchestration

- Transformations rely on orchestration as the "pump" that coordinates discrete operations across multiple systems.
- Orchestration allows for the assembly of complex data pipelines that span multiple storage and query engines.

ETL vs. ELT

❖ Shifting Paradigms

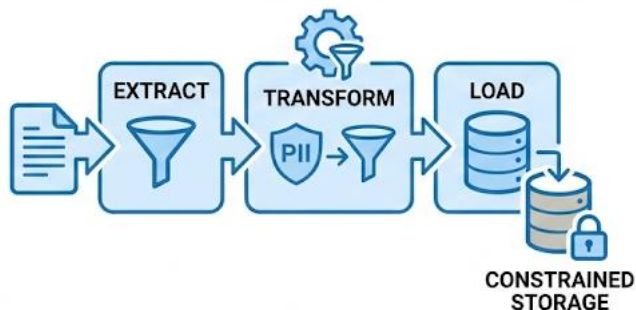
- Traditional ETL (Extract-Transform-Load) was driven by the resource constraints of older on-premises target systems.
- ELT (Extract-Load-Transform) loads raw data into a warehouse or lake first, utilizing the target's power for the transformation.

❖ The Data Lake Context

- In data lakes, ELT can lead to "data swamps" if data is loaded without a clear plan for future transformation.
- Modern lakehouse environments often blur the lines between ETL and ELT as data moves fluidly between object storage and metadata layers.

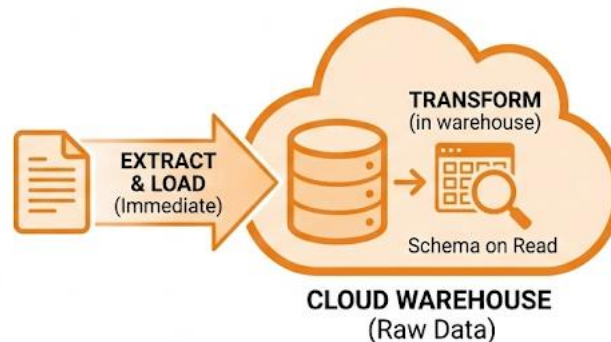
ETL vs. ELT

ETL (Extract-Transform-Load)



Transform data before storage. Essential for PII masking or when storage is constrained.

ELT (Extract-Load-Transform)



Load raw data immediately, then transform in the warehouse. Favored in cloud for auditability ('Schema on Read').

Transformation Tools and Languages

❖ SQL as a Declarative Powerhouse

- SQL is a first-class citizen in big data; tools like *dbt* allow analysts to build complex, version-controlled pipelines using SQL.
- While declarative, SQL can build complex DAGs and transformations through scripts and orchestration tools.

❖ Code-Based Frameworks

- Native code (Python, Java, Scala) is better for complex procedural logic like word stemming or custom libraries.
- These frameworks require more optimization finesse from the engineer, such as manual join reordering and UDF management.

Update Patterns in Transformations

❖ Truncate and Reload

- Wipes old data and replaces it entirely, which is simple but inefficient for large sets.

❖ Insert only

- Inserts new records without changing or deleting old records.

❖ Delete

- (Hard) Deletion is critical when a source system deletes data and satisfies recent regulatory changes. In columnar systems and data lakes, deletes are more expensive than inserts.

❖ Upsert/Merge

- Upserting matches source records to a target table, updating existing ones and inserting new ones.
- In columnar systems, this is expensive because of "Copy on Write" (COW), where changing one record requires rewriting the entire data block.

Data Harmonization

❖ Data Harmonization - A Single Source of Truth

- Data harmonization is the process of unifying disparate data from various sources, formats, and structures into a consistent, standardized, and cohesive dataset. It ensures semantic consistency using a "common language" for all information.

❖ Moving from Chaos to Clarity

- Organizations often have fragmented data scattered across silos with different fields, units of measure, and definitions.
- Harmonization transforms this "data swamp" into a workable, accurate body of information that provides a holistic view of the business.

Key Steps in Harmonization

❖ Identify and Map

- Locate all relevant data sources (internal databases, SaaS apps, etc.) and create a unified target data model, mapping source fields to this new structure.

❖ Cleanse and Transform

- Standardize formats (e.g., dates, currencies), convert units of measurement, resolve semantic inconsistencies, and remove errors or duplicates.

❖ Validate and govern

- Test the harmonized data for accuracy and integrity.
- Implement ongoing governance to maintain quality as new data sources are added.

Challenges and Real-World Use Cases

❖ Common Challenges

- Data Diversity and Complexity: Managing vast differences in structures, formats, and semantics across numerous, ever-growing data sources is a significant hurdle.
- Maintaining Quality and Governance: Establishing robust processes to ensure ongoing data accuracy, consistency, and compliance prevents "data drift" over time.

❖ Some use cases

- Healthcare and Research: Combining patient data from disparate clinical trials or merging global health data to track disease patterns and accelerate drug development.
- Marketing and Customer Experience: Creating a 360-degree view of the customer by blending data from CRM, e-commerce, and marketing platforms to personalize interactions.
- Supply Chain and Logistics: Harmonizing data from various suppliers, partners, and logistics providers to improve visibility, track shipments, and optimize operations.

Data Wrangling and Business Logic

- ❖ Data wrangling takes malformed, unstructured data and parses it into a useful, clean format.
 - Modern GUI-based tools act as IDEs for data, allowing engineers to visually identify anomalies and outliers.
- ❖ Derived Data and Business Logic
 - Derived data is computed from other data to encode complex logic like profit-after-marketing-costs.
 - Because business logic changes, engineers may use a "metrics layer" to centralize definitions and ensure consistency across reports.

Standard Database Views

❖ Definition and Execution Mechanisms

- A view is a database object that acts essentially as a saved query, allowing users to select from it just like a regular table.
- At runtime, the database combines the user's query with the view's underlying subquery, then optimizes and runs the full combined query.

❖ Strategic Utility of Views

- Views enhance security by acting as a filter, restricting access to specific columns or rows based on user roles.
- They simplify complex access patterns by pre-joining multiple tables into a single "wide" table presentation for easier downstream analysis.

Materialized Views

❖ Precomputation for Performance Gains

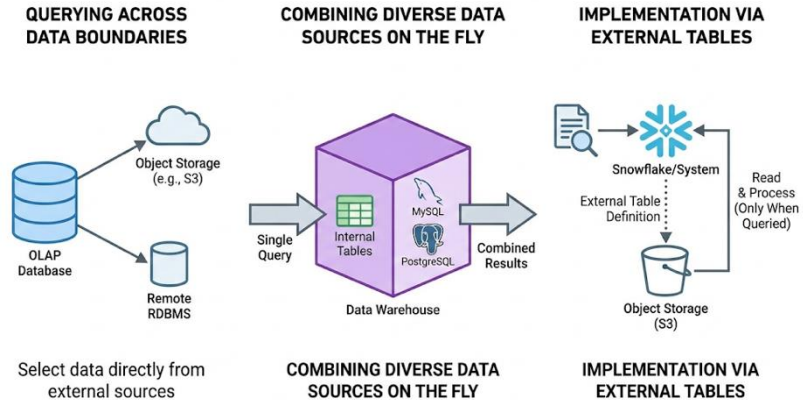
- Unlike standard views, a materialized view computes previously defined complex logic in advance and persists the result set.
- When a user references the view, they query directly from pre-joined, pre-computed data rather than triggering expensive operations at runtime.

❖ Automated Management and Optimization

- The database manages the execution and updates the materialized view results whenever changes occur in the underlying source tables.
- Advanced query optimizers can identify standard queries that "look like" a materialized view and automatically rewrite them to use the precomputed results.

Federated Queries

- ❖ Querying Across Data Boundaries
 - Federated queries enable an OLAP database to select data directly from external sources, such as object storage or remote RDBMSs.
- ❖ Combining Diverse Data Sources on the Fly
 - A data warehouse can issue a single query that returns combined results from internal tables and external sources like MySQL or PostgreSQL.
- ❖ Implementation via External Tables
 - Systems like Snowflake use "external tables" defined on object storage (e.g., S3), where data is read and processed based on defined parameters only when queried.



Data Virtualization

❖ Processing Without Internal Storage

- Data virtualization entails using a data processing query engine, such as Trino or Presto, that does not store data internally but queries disparate sources.

❖ Optimizing Performance with Query Pushdown

- Virtualization engines aim to move computational work, like filtering predicates, to the source databases to offload computation and reduce network traffic.

❖ Abstracting Organizational Silos

- Virtualization serves as a tool to expand the data lake by abstracting away barriers between data silos, often serving as an access layer in Data Mesh architectures.

Transformation Undercurrents

❖ Security and Management

- Transformation creates new datasets that require fine-grained access control at the row, column, and cell levels.
- Data management must track "lineage" to help users understand exactly where data came from and how it was modified.

❖ DataOps and Observability

- Engineers must monitor "data health," checking for statistical distortions and statistical data downtime.
- Operational monitoring includes tracking query queue length, memory usage, and the cost-efficiency of queries (FinOps).

Advanced Transformation Architectures

❖ Optimizing Distributed Transformation Engines

- Distributed query engines handle massive scale by utilizing techniques like broadcast joins for efficiency or resource-intensive shuffle hash joins when repartitioning is necessary.

❖ Managing State in Columnar Storage

- Because modern columnar databases are optimized for reads, handling updates requires specific patterns like "Insert Only" appends or computationally expensive "Upsert/Merge" operations that rewrite files.

❖ The Rise of Composable Data Flows

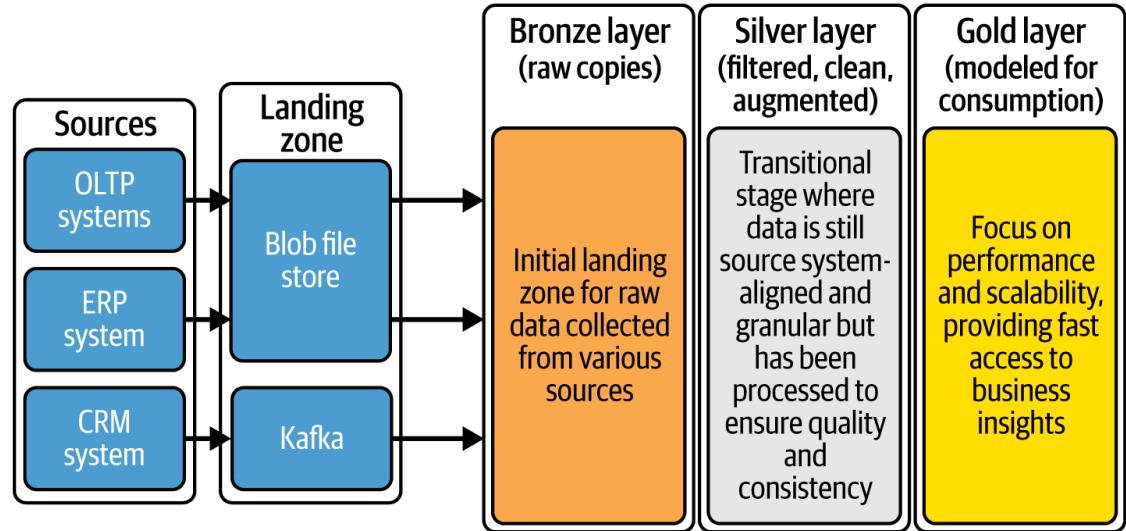
- Emerging tools support composable materialized views, allowing data to flow asynchronously down a chain of tables that update continuously as new data arrives.

❖ Abstracting Boundaries via Virtualization

- Data virtualization techniques connect disparate sources to expand the data lake concept, often utilizing query pushdown to offload computation back to source systems to prevent network bottlenecks.

The Medallion architecture

- ❖ A design pattern used in lakehouses to progressively improve data maturity by adding structure, quality, and business context at each stage.
- ❖ It organizes data into:
 - The **Bronze Layer**:
 - Native Foundation
 - The **Silver Layer**:
 - Cleansed, Standardized Data
 - The **Gold Layer**:
 - Refined Business Assets



<https://learning.oreilly.com/library/view/building-medallion-architectures/9781098178826/ch03.html#id32>

The Medallion architecture - The Bronze Layer

❖ Raw Data Preservation

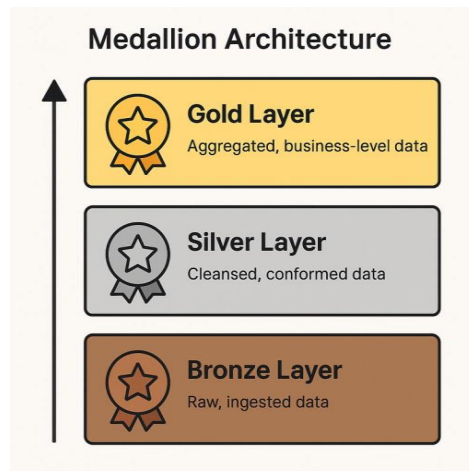
- The Bronze layer represents the starting point of data maturity, where information is stored in its **raw, unaltered format** directly from source systems.

❖ Single Source of Truth

- It acts as a comprehensive repository that maintains the "original fidelity" of data, providing a foundation from which all downstream refined datasets flow.

❖ Data Ingestion

- While this layer often contains **unfiltered data with quality issues**, it allows organizations to ingest petabytes of structured and unstructured information at high velocity.



The Medallion architecture - The Silver Layer

❖ Quality Improvement

- In the Silver layer, data maturity increases as engineers focus on **initial cleansing, reformatting, and persistence**.

❖ Schema Enforcement

- This layer adds structure by **schematizing content and enforcing technical constraints**, ensuring data types are correctly evaluated for downstream use.

❖ Analytical Readiness

- By providing cleansed and enriched datasets, the Silver layer creates a stable foundation for building **machine learning models and complex analytical workloads**.

The Medallion architecture - The Gold Layer

❖ User-Centric Delivery

- The Gold layer represents the highest level of maturity for specific use cases, exposing data in **refined formats expected by end users**, such as data marts or reference datasets.

❖ Performance Optimization

- Data at this stage is often **aggregated or summarized** to ensure faster query performance in business intelligence (BI) tools and dashboards.

❖ Business Alignment

- This layer moves away from technical standardization toward **business-aligned deliverables**, satisfying the specific reporting and decision-making requirements of the organization.

Improving Organizational Data Maturity

❖ Lifecycle Management

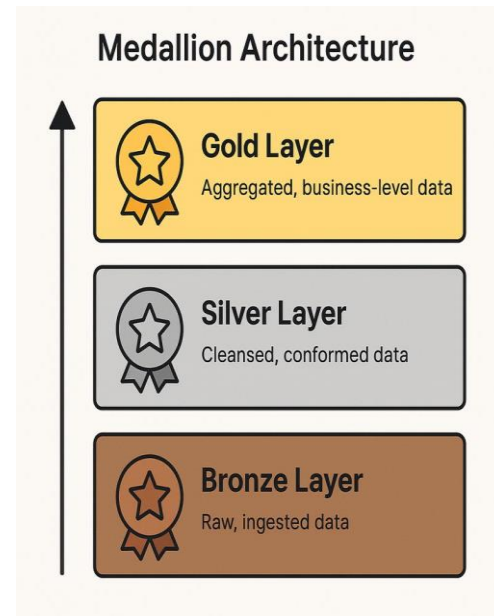
- Implementing these layers helps an organization progress through maturity stages, from **"Starting with data"** by building a foundation, to **"Leading with data"** through the delivery of high-value automated insights.

❖ Governance and Trust

- The architecture improves maturity by incorporating **governance, auditing, and fine-grained access controls** directly into the data flow.

❖ Operational Efficiency

- By separating technical cleansing from business transformations, teams can **reduce development effort** and focus on implementing the logic that drives strategic value.



Pros and Cons

- ❖ Data Quality & Trust
 - The tiered approach provides clear, consistent datasets for analysis.
- ❖ Clear Lineage & Auditing
 - The bronze layer serves as a reliable source for reprocessing, simplifying audits.
- ❖ Scalability
 - It handles both batch and streaming data efficiently.
- ❖ Modular Design
 - Allows data teams to maintain and debug, or transform data, in stages.
- ❖ Increased Storage Costs
 - Storing data at three different stages triples storage usage.
- ❖ Higher Complexity
 - Managing multiple pipelines and data sets requires more effort.
- ❖ Latency
 - The multi-stage transformation process can introduce delays between raw ingestion and insights.
- ❖ Overhead for Small Projects
 - It is not ideal for small-scale projects, as it might add unnecessary steps.

Transformations Summary

❖ The Heart of the Lifecycle

- Transformations are where data engineering creates tangible ROI by preparing data for analytics and ML.
- Engineers must balance the use of exciting new tools with the fundamental goal of serving stakeholders accurately.

❖ Looking Forward

- The future of transformations lies in the "live data stack," bringing processing closer to real-time applications.
- Continuous learning and stakeholder feedback loops are essential for maintaining a high-quality transformation layer.

Further Reading & Resources

- ❖ Reis & Housley, Fundamentals of Data Engineering,
 - Chapter 8
 - <https://learning.oreilly.com/library/view/fundamentals-of-data/9781098108298/>
- ❖ Piethein Strengholt, Building Medallion Architectures, 2025.
 - <https://learning.oreilly.com/library/view/building-medallion-architectures/9781098178826/>
 - Chapters 1-3 (... *a must read*)

